

Aprendizaje Automático (Machine Learning)

Técnicas de Inteligencia Artificial

Prof. Flavio Prieto
email: faprieto@unal.edu.co

INGENIERÍA MECATRONICA
Facultad de Ingeniería
Universidad Nacional de Colombia Sede Bogotá



23 de abril de 2026

Aprendizaje por Refuerzo.

- ▶ El Aprendizaje por Refuerzo (RL) es un paradigma donde un agente (entidad) **aprende interactuando con un entorno**.
- ▶ El agente no recibe instrucciones explícitas, sino que **explora acciones y recibe recompensas** como retroalimentación.
- ▶ **El objetivo es maximizar la recompensa acumulada** a lo largo del tiempo.

Agente

- ▶ Se denomina **agente** a la entidad que toma decisiones dentro de un entorno con el objetivo de maximizar una medida acumulada de recompensa.
- ▶ **Características:**
 - ▶ Observa el estado del entorno: s_t
 - ▶ Selecciona acciones: a_t
 - ▶ Aprende a partir de la interacción
- ▶ **Función del agente:**

$$a_t = \pi(s_t)$$

donde π representa la política del agente.

Entorno

- ▶ El **entorno** es todo aquello con lo que el agente interactúa.
- ▶ **Dinámica del entorno:**

$$s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$$

- ▶ Elementos clave:
 - ▶ Estados: conjunto $\mathcal{S}$
 - ▶ Acciones: conjunto $\mathcal{A}$
 - ▶ $P(s_{t+1} \mid s_t, a_t)$ probabilidad de que el entorno pase al estado s_{t+1} dado que el agente estaba en el estado s_t y ejecutó la acción a_t .
- ▶ El entorno responde a las acciones del agente generando nuevos estados y recompensas.

Política

- ▶ La **política** define el comportamiento del agente, es decir, cómo selecciona acciones dado un estado.

- ▶ **Tipos de políticas:**

- ▶ Determinística:

$$a = \pi(s)$$

- ▶ Estocástica:

$$\pi(a | s) = P(a | s)$$

- ▶ **Objetivo:**

Encontrar una política óptima π^* tal que maximice el retorno esperado.

Recompensa

- ▶ La **recompensa** es una señal escalar que indica la calidad inmediata de una acción.

$$r_t = R(s_t, a_t)$$

- ▶ **Interpretación:**
 - ▶ Valores positivos: acciones deseables
 - ▶ Valores negativos: penalizaciones
- ▶ **Importancia:** La función de recompensa guía el aprendizaje del agente.

Retorno

- ▶ El **retorno** es la suma acumulada de recompensas futuras.
- ▶ **Retorno descontado:**

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

- ▶ **Parámetro de descuento:**
 - ▶ $\gamma \in [0, 1]$
 - ▶ Controla la importancia del futuro
- ▶ **Casos extremos:**
 - ▶ $\gamma = 0$: solo importa la recompensa inmediata
 - ▶ $\gamma \approx 1$: se consideran recompensas a largo plazo

Modelo de Aprendizaje por Refuerzo.

Un problema de RL se modela como un Proceso de Decisión de Markov (MDP)¹:

$$\mathcal{M} = (S, A, P, R, \gamma)$$

- ▶ **S: Conjunto de estados** del entorno.
- ▶ **A: Conjunto de acciones** que el agente puede tomar.
- ▶ $P(s_{t+1}|s_t, a)$: **Probabilidad de transición** al estado s_{t+1} dado que el agente estaba en s_t y tomó la acción a .
- ▶ $R(s_t, a)$: **Recompensa inmediata** obtenida al ejecutar a en s_t .
- ▶ γ : **Factor de descuento** ($0 \leq \gamma \leq 1$), que pondera la importancia de recompensas futuras.

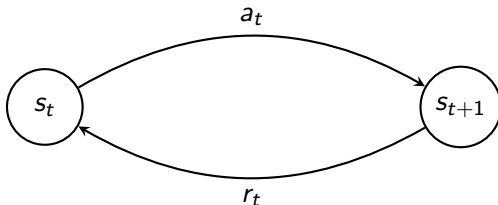
¹Un Modelo de Decisión de Markov es un marco matemático usado para modelar toma de decisiones secuenciales en situaciones donde los resultados son parcialmente aleatorios y parcialmente controlados por un agente.

Dinámica del Agente.

El agente interactúa de manera iterativa:

1. El agente observa un estado s_t .
2. Selecciona una acción a_t según una política $\pi(a|s)$.
3. El entorno devuelve:
 - ▶ Una recompensa r_t .
 - ▶ Un nuevo estado s_{t+1} .

Este ciclo se repite hasta alcanzar un estado terminal o indefinidamente.



Función de Retorno

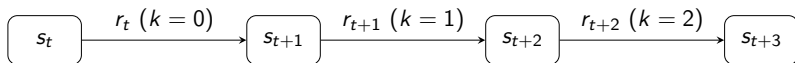
El retorno en el instante t se define como la suma descontada de las recompensas futuras, o recompensa acumulada esperada:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

donde:

- ▶ r_{t+k} es la recompensa obtenida en el paso $t + k$,
- ▶ $\gamma \in [0, 1]$ es el factor de descuento, pondera la importancia de recompensas futuras frente a inmediatas,
- ▶ k es un **índice de desplazamiento temporal**.

El objetivo del agente es maximizar G_t .

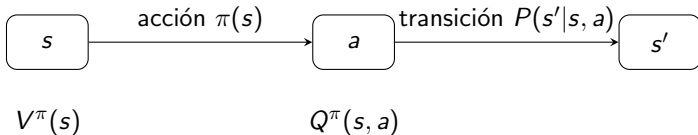


Funciones de Valor.

Para evaluar qué tan buena es una política π , se definen funciones de valor:

- ▶ **Valor de estado:** $V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$
- ▶ **Valor de acción:** $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$
 - ▶ $\mathbb{E}_\pi[\cdot]$: esperanza matemática calculada **asumiendo que las acciones futuras se eligen siguiendo la política π .**
 - ▶ $V^\pi(s)$: valor esperado del retorno al iniciar en s y seguir π .
 - ▶ $Q^\pi(s, a)$: valor esperado del retorno al iniciar en s , tomar acción a , y luego seguir π .

Estas funciones guían al agente hacia mejores decisiones.



Objetivo del Aprendizaje por Refuerzo.

El objetivo del RL es **encontrar una política óptima** π^* que maximice el retorno esperado acumulado a lo largo del tiempo:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid S_t = s \right]$$

o equivalentemente:

$$\pi^* = \arg \max_{\pi} V^{\pi}(s), \quad \forall s \in S$$

donde:

- ▶ π es una política (determinística o estocástica).
- ▶ $\gamma \in [0, 1]$ es el factor de descuento.
- ▶ r_{t+k} es la recompensa en el paso $t + k$.
- ▶ La expectativa $\mathbb{E}_{\pi}[\cdot]$ se calcula suponiendo que las acciones se eligen siguiendo la política π .

► Función objetivo en términos de valor de estado:

Equivalente a lo anterior, el objetivo es encontrar una política tal que **el valor esperado de cada estado inicial se maximice**:

$$V^{\pi^*}(s) = \max_{\pi} V^{\pi}(s), \quad \forall s \in \mathcal{S}$$

► Función objetivo con función de acción-valor:

Otra forma de expresarlo es mediante la función $Q^{\pi}(s, a)$:

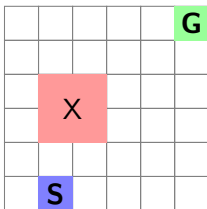
$$Q^{\pi^*}(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

y la política óptima, que selecciona la acción con mayor valor esperado, es:

$$\pi^*(s) = \arg \max_a Q^{\pi^*}(s, a)$$

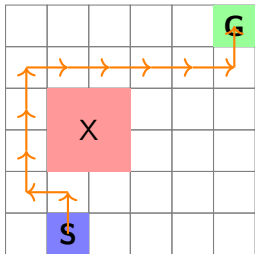
Ejemplo: Un robot en un laberinto debe moverse de la posición **S** a la posición **G**

- ▶ **Estados (S):** posiciones en el laberinto.
- ▶ **Acciones (A):** moverse arriba, abajo, izquierda, derecha.
- ▶ **Recompensas (R):**
 - ▶ +10 al llegar a la meta (**G**),
 - ▶ -1 por cada paso.
- ▶ **Objetivo:** encontrar una política π que lo lleve a la salida minimizando pasos.



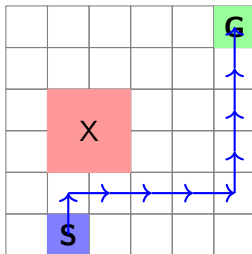
Ejemplo de Política Óptima vs Subóptima:

Política Subóptima



- ▶ Rodeo por la izquierda.
- ▶ Más pasos (11) $\Rightarrow$ menor retorno acumulado:
 $G = -11$.

Política Óptima π^*



- ▶ Encuentra un camino más corto.
- ▶ Menos pasos (9) $\Rightarrow$ mayor retorno acumulado:
 $G = -9$.

Estrategias generales en RL

- ▶ Corresponden a los enfoques fundamentales que determinan cómo el agente aprende a partir de su interacción con el entorno.
- ▶ No representan algoritmos específicos, sino marcos conceptuales que orientan el diseño y desarrollo de métodos de aprendizaje.

- ▶ **Métodos libres de modelo (Model-free RL):** el agente aprende directamente una política o función de valor a partir de la experiencia, sin modelar explícitamente la dinámica del entorno.
- ▶ **Métodos basados en modelos (Model-based RL):** el agente aprende o utiliza un modelo de la dinámica del entorno (transiciones y recompensas) para planificar y tomar decisiones.
- ▶ **Estrategias híbridas:** combinan aprendizaje a partir de experiencia directa con el uso parcial de modelos o distintas formas de actualización (por ejemplo, basadas en episodios completos o en transiciones individuales).
- ▶ **Aprendizaje profundo por refuerzo (Deep RL):** incorpora redes neuronales profundas para aproximar funciones de valor o políticas, permitiendo manejar espacios de estados y acciones de alta dimensión.

Métodos libres de modelo.

- ▶ El agente no intenta aprender la dinámica del entorno, sino directamente la política o las funciones de valor.
 - ▶ **Monte Carlo (MC):** actualizan funciones de valor usando recompensas al final de episodios completos.

$$V(s_t) \leftarrow V(s_t) + \alpha(G_t - V(s_t))$$

- ▶ **Diferencia Temporal (TD):** actualizan valores de forma incremental usando transiciones parciales.

$$V(s_t) \leftarrow V(s_t) + \alpha(r + \gamma V(s_{t+1}) - V(s_t))$$

- ▶ **Ejemplos:** Q-learning, SARSA, Actor-Critic.

Métodos basados en modelos.

- ▶ El agente trata de aprender una representación explícita de la dinámica del entorno:

$$P(s_{t+1} \mid s_t, a), \quad R(s_t, a)$$

- ▶ Una vez aprendido el modelo, puede planificar y simular futuros estados.
- ▶ **Ventaja:** más *sample-efficient* (requieren menos interacción).
- ▶ **Ejemplo:** Dyna-Q (mezcla entre planificación y aprendizaje).

Estrategias híbridas.

- ▶ Combinan modelos del entorno con métodos libres de modelo.
- ▶ **Ejemplo:** usar simulación de modelos aprendidos para acelerar aprendizaje de políticas (Model-based + Deep RL).

Aprendizaje profundo por refuerzo (Deep RL)

- ▶ Uso de redes neuronales para aproximar funciones de valor, políticas o ambos.
- ▶ Aplica a cualquier categoría anterior.
- ▶ Permite manejar espacios de estados y acciones de alta dimensión.
- ▶ **Ejemplos:**
 - ▶ **Deep Q-Network (DQN):** función $Q(s, a; \theta)$.
 - ▶ **Policy Gradient profundo:** $\pi(a | s; \theta)$.

Métodos principales de aprendizaje de políticas.

Se refieren a los enfoques fundamentales que definen qué aprende el agente y cómo toma decisiones.

Se clasifican principalmente en dos grandes familias (a veces complementadas con híbridos):

- ▶ **Métodos basados en valor (Value-based methods):**
Aprenden *qué tan buenos* son los estados o acciones.
- ▶ **Métodos basados en política (Policy-based methods):**
Aprenden directamente *cómo actuar*.
- ▶ **Métodos híbridos (Actor-Critic methods):**
Integran ambos enfoques para mayor eficiencia y estabilidad.

Métodos basados en valor.

- ▶ **Idea central:** el agente aprende funciones de valor y a partir de ellas deriva la política.
- ▶ Funciones principales:
 - ▶ **Valor de estado:**

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid S_t = s \right]$$

- ▶ **Valor de acción (Q-función):**

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid S_t = s, A_t = a \right]$$

- ▶ La política π suele obtenerse mediante selección ϵ -greedy sobre $Q(s, a)$.
- ▶ **Ejemplos:** Q-learning, SARSA, Deep Q-Networks (DQN).

Métodos basados en política.

- ▶ **Idea central:** el agente aprende directamente una política parametrizada:

$$\pi_{\theta}(a \mid s) = P(A = a \mid S = s; \theta)$$

- ▶ Se optimiza θ maximizando la recompensa esperada:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}[G_t]$$

- ▶ El gradiente de política se estima con:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[G_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) \right]$$

- ▶ **Ventajas:**

- ▶ Manejan directamente políticas estocásticas.
- ▶ Funcionan bien en espacios de acción continuos.

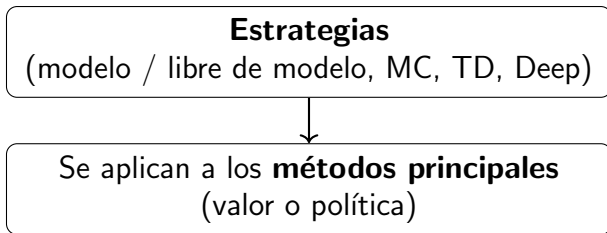
- ▶ **Ejemplos:** REINFORCE, Policy Gradient, Proximal Policy Optimization (PPO).

Métodos híbridos.

- ▶ **Idea central:** combinan lo mejor de ambos mundos:
 - ▶ **Actor:** actualiza la política directamente.
 - ▶ **Critic:** estima funciones de valor para guiar al actor.
- ▶ Reducen varianza respecto a REINFORCE y mejoran estabilidad.
- ▶ **Ejemplos:** Advantage Actor-Critic (A2C), Deep Deterministic Policy Gradient (DDPG).

Relación general

- ▶ Las **estrategias generales** (modelo, libre de modelo, MC, TD, deep) definen *cómo se aprende*.
- ▶ Los **métodos principales** (valor vs política) definen *qué se aprende*.
- ▶ Ambos ejes son ortogonales y se combinan en la práctica.



Algoritmo ϵ -greedy.

- ▶ Estrategia de selección de acciones utilizada en aprendizaje por refuerzo.
- ▶ Permite equilibrar **exploración** (probar acciones nuevas) y **explotación** (usar la acción que actualmente parece mejor).
- ▶ No es un algoritmo de aprendizaje por sí mismo, sino una **política de decisión**.
 - ▶ Con probabilidad ϵ se elige una acción aleatoria: **exploración**.
 - ▶ Con probabilidad $1 - \epsilon$ se elige la acción con el mayor valor estimado: **explotación**.
 - ▶ Fórmula:

$$a_t = \begin{cases} \text{acción aleatoria,} & \text{con probabilidad } \epsilon \\ \arg \max_a Q(s_t, a), & \text{con probabilidad } 1 - \epsilon \end{cases}$$

Características del algoritmo ϵ -greedy.

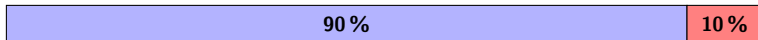
- ▶ Permite controlar la exploración mediante el parámetro ϵ .
- ▶ Muy fácil de implementar con un generador de números aleatorios.
- ▶ Compatible con métodos basados en valor como Q-learning, SARSA, Dyna-Q.
- ▶ La exploración puede disminuir gradualmente disminuyendo ϵ con el tiempo (*epsilon decay*).

Ejemplo numérico de ϵ -greedy.

- Supongamos un estado s con tres acciones posibles:

Acción	$Q(s, a)$
A	5
B	3
C	4

- Si $\epsilon = 0.1$:
 - **Exploración:** 10 % del tiempo se elige una acción al azar entre A, B o C.
 - **Explotación:** 90 % del tiempo se elige la acción con mayor Q , que es la acción A.



Explotación
A (mejor Q)

Exploración
B o C (aleatoria)

Métodos de Aprendizaje por Refuerzo.

Existen diversos métodos de aprendizaje por refuerzo, cada uno con características y aplicaciones particulares.

- ▶ **Métodos de Aprendizaje:** Q-learning, SARSA, DQN, etc.
- ▶ **Tipo y Espacio de Acción:** Basado en valor, Basado en política, Discreto, Continuo
- ▶ **Forma de Actualización:** Q-learning: TD, SARSA: TD, Policy Gradient: Gradiente, Actor-Critic: TD + Gradiente
- ▶ **Aplicaciones:** Videojuegos, Robótica, Control de sistemas

- ▶ En las tablas siguientes se resume:
 1. **Tipo y espacio de acción** de algunos métodos.
 2. **Forma de actualización** de los valores o políticas durante el aprendizaje.
 3. **Aplicaciones típicas** donde se usan estos métodos.

Métodos de Aprendizaje por Refuerzo.

Método	Tipo	Espacio de acción
Q-learning	Basado en valor	Discreto
SARSA	Basado en valor	Discreto
DQN	Basado en valor + Deep RL	Discreto (alto dimensional)
Dyna-Q	Basado en valor híbrido	Discreto
REINFORCE	Basado en política	Discreto o continuo
Policy Gradient	Basado en política	Discreto o continuo
PPO	Basado en política + Deep RL	Discreto o continuo
Actor-Critic	Híbrido	Discreto o continuo
A2C / A3C	Híbrido	Discreto o continuo
DDPG	Actor-Critic profundo	Continuo
SAC	Actor-Critic profundo	Continuo

Método	Forma de actualización
Q-learning	Actualiza la Q-función usando la máxima recompensa futura: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
SARSA	Actualiza la Q-función usando la acción realmente tomada: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$
DQN	Actualiza la Q-función aproximada con red neuronal y experiencia replay
Dyna-Q	Combina actualización directa de Q y planificación usando un modelo aprendido del entorno
REINFORCE	Actualiza la política directamente usando gradiente de la recompensa: $\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a s) R$
Policy Gradient	Actualiza los parámetros de la política siguiendo el gradiente esperado de recompensa
PPO	Actualiza la política usando gradiente de probabilidad ratio con clipping para estabilidad
Actor-Critic	Actor se actualiza vía gradiente de política, crítico usa TD-error: $\delta = r + \gamma V(s') - V(s)$
A2C / A3C	Similar a Actor-Critic, usando múltiples agentes en paralelo para estabilidad
DDPG	Actor se actualiza usando gradiente determinista, crítico usa TD-error continuo
SAC	Actor y crítico actualizados usando gradientes suavizados con entropía para exploración

Método	Aplicaciones típicas
Q-learning	Juegos de tablero simples, control discreto de robots
SARSA	Tareas de navegación con riesgo, simulaciones discretas
DQN	Videojuegos, entornos con alta dimensionalidad de estados
Dyna-Q	Simulación y planificación en entornos discretos
REINFORCE	Control continuo simple, políticas estocásticas
Policy Gradient	Control continuo, robótica, optimización de políticas
PPO	Videojuegos, simulaciones robóticas, entornos continuos
Actor-Critic	Control de robots, simulaciones de sistemas dinámicos
A2C / A3C	Tareas de robótica, entornos complejos paralelos
DDPG	Control continuo de robots, manipulación de brazos robóticos
SAC	Control de robots, locomoción, entornos continuos complejos

Q-Learning.

Q-Learning^a es un algoritmo popular de **aprendizaje por refuerzo sin modelo (model-free)** que permite a un agente aprender a tomar las mejores decisiones interactuando con su entorno.

- ▶ El agente **no necesita un modelo del entorno**.
- ▶ Aprende **directamente de la experiencia**.
- ▶ Explora diferentes acciones y observa sus resultados.

^aDiapositivas basadas en: [Q-Learning in Reinforcement Learning](#).

Idea intuitiva:

- ▶ Un sistema observa una manzana pero dice incorrectamente: *“Es un mango”*.
- ▶ Recibe retroalimentación: *“Incorrecto, es una manzana”*.
- ▶ El sistema aprende de ese error.
- ▶ La próxima vez que vea la manzana, responderá correctamente: *“Es una manzana”*.

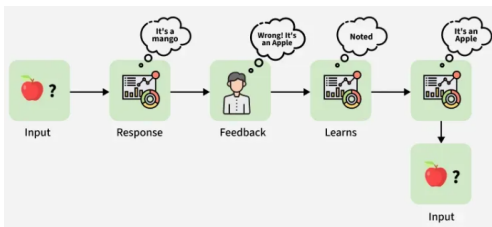


Figura tomada de: [Internet](#).

Este proceso de **prueba y error guiado por retroalimentación** es la base del funcionamiento de **Q-Learning**.

- ▶ La idea central de **Q-Learning** es que el agente construye una **tabla Q (Q-table)** que almacena **valores Q**.
- ▶ Cada valor Q estima qué tan conveniente es ejecutar una acción específica en un estado determinado, considerando la **recompensa futura esperada**.
- ▶ Con el tiempo, el agente **actualiza esta tabla** utilizando la retroalimentación obtenida al interactuar con el entorno.

Componentes Clave de Q-Learning

1. Valores Q (Action-Values)

- ▶ Los valores Q representan la **recompensa esperada** al ejecutar una acción en un estado determinado.
- ▶ Estos valores se actualizan progresivamente usando la **regla de actualización de Diferencia Temporal (Temporal Difference, TD)**.

2. Recompensas y Episodios

- ▶ El agente se mueve entre diferentes estados ejecutando acciones.
- ▶ Después de cada acción recibe una **recompensa**.
- ▶ El proceso continúa hasta alcanzar un **estado terminal**, que marca el final del **episodio**.

Actualización por Diferencia Temporal (TD)

- ▶ El agente actualiza los valores Q utilizando la siguiente ecuación:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma Q(S', A') - Q(S, A) \right)$$

Donde:

- ▶ S : estado actual.
- ▶ A : acción ejecutada por el agente.
- ▶ S' : siguiente estado alcanzado.
- ▶ A' : mejor acción posible en el estado S' .
- ▶ R : recompensa obtenida al ejecutar la acción A en S .
- ▶ γ : **factor de descuento**, que equilibra recompensas inmediatas y futuras.
- ▶ α : **tasa de aprendizaje**, que determina cuánto influye la nueva información en los valores Q previos.

Política ϵ -Greedy

La política ϵ -**greedy** permite decidir qué acción tomar utilizando las estimaciones actuales de los valores Q .

Explotación

- ▶ Con probabilidad $1 - \epsilon$, el agente selecciona la acción con el **mayor valor Q** .
- ▶ Utiliza el conocimiento actual para **maximizar la recompensa**.

Exploración

- ▶ Con probabilidad ϵ , el agente elige una **acción aleatoria**.
- ▶ Esto permite **explorar nuevas posibilidades** y descubrir mejores estrategias.

La combinación de exploración y explotación permite al agente **mejorar su política de decisión con el tiempo**.

Funcionamiento de Q-Learning

- ▶ El algoritmo **Q-Learning** sigue un proceso iterativo donde varios componentes interactúan para entrenar al agente.
- ▶ El procedimiento general se muestra en la figura.

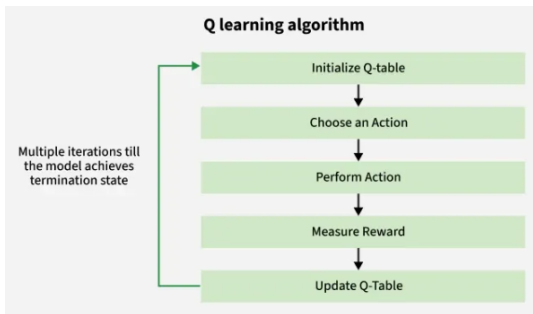


Figura tomada de: [Internet](#).

1. Inicio en un estado (S)

- ▶ El entorno proporciona al agente un **estado inicial** s .
- ▶ Este estado describe la **situación actual del sistema**.
- ▶ El objetivo del agente es aprender la función:

$$Q(s, a)$$

donde $Q(s, a)$ representa la **recompensa acumulada esperada** al ejecutar la acción a en el estado s .

2. Selección de una acción (A)

- ▶ El agente selecciona una acción según su **política**.
- ▶ La decisión se basa en los valores almacenados en la **tabla Q**.
- ▶ Usualmente se emplea una política **ϵ -greedy**:

$$a = \begin{cases} \text{acción aleatoria} & \text{con probabilidad } \epsilon \\ \arg \max_a Q(s, a) & \text{con probabilidad } 1 - \epsilon \end{cases}$$

3. Ejecución de la acción y respuesta del entorno

- ▶ El agente ejecuta la acción seleccionada.
- ▶ El entorno responde proporcionando:
 - ▶ un nuevo estado s'
 - ▶ una recompensa r
- ▶ La experiencia obtenida puede representarse como:

$$(s, a, r, s')$$

4. Actualización de la tabla Q

- ▶ El agente actualiza el valor asociado al par estado–acción.
- ▶ Regla de actualización de **Q-Learning**:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

donde:

- ▶ α : tasa de aprendizaje
- ▶ γ : factor de descuento

5. Refinamiento de la política

- ▶ A medida que los valores $Q(s, a)$ se actualizan, el agente mejora su política.
- ▶ El ciclo se repite continuamente:
 - ▶ observar estados,
 - ▶ ejecutar acciones,
 - ▶ recibir recompensas,
 - ▶ actualizar los valores Q .
- ▶ La política óptima se obtiene seleccionando la acción con mayor valor:

$$\pi(s) = \arg \max_a Q(s, a)$$

- ▶ Después de muchos episodios de interacción con el entorno, el agente aprende una **política óptima** que maximiza la recompensa acumulada.

Métodos para Determinar los Valores Q

1. Diferencia Temporal (Temporal Difference, TD)

- ▶ La Diferencia Temporal se calcula comparando los valores actuales de estado–acción con los valores previamente estimados.
- ▶ Permite que el agente **aprenda directamente de la experiencia**.
- ▶ No requiere un **modelo explícito del entorno**.

El error de Diferencia Temporal se define como:

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

donde δ mide la diferencia entre el valor estimado y el valor observado.

2. Ecuación de Bellman

- ▶ Es una fórmula recursiva utilizada para calcular el valor de un estado y determinar la acción óptima.
- ▶ Es un elemento fundamental en el contexto de **Q-Learning**.
- ▶ La ecuación de Bellman para la función acción–valor es:

$$Q(s, a) = R(s, a) + \gamma \max_a Q(s', a)$$

Donde:

- ▶ $Q(s, a)$: valor Q para el par estado–acción.
- ▶ $R(s, a)$: recompensa inmediata al ejecutar la acción a en el estado s .
- ▶ γ : factor de descuento que representa la importancia de las recompensas futuras.
- ▶ $\max_a Q(s', a)$: valor Q máximo del siguiente estado s' considerando todas las acciones posibles.

Q-table

- ▶ La **Q-table** es una estructura de memoria donde el agente almacena información sobre qué acciones generan las mejores recompensas en cada estado.
 - ▶ Es una tabla de **valores Q**.
 - ▶ Representa el **conocimiento del agente sobre el entorno**.
 - ▶ Se actualiza continuamente a medida que el agente interactúa con el entorno.
- ▶ La Q-table permite que el agente tome **decisiones informadas**, identificando las acciones que probablemente conduzcan a mayores recompensas.

Estructura de una Q-table

- ▶ Las **filas** representan los **estados**.
- ▶ Las **columnas** representan las **acciones posibles**.
- ▶ Cada celda contiene el **valor Q** correspondiente al par estado–acción.

A medida que el agente aprende mediante **exploración y explotación**, los valores de la tabla se refinan, reflejando las mejores acciones para cada estado y conduciendo a una **toma de decisiones óptima**.

- ▶ Las **filas** representan los **estados**.
- ▶ Las **columnas** representan las **acciones posibles**.
- ▶ Cada celda contiene el **valor Q** correspondiente al par estado–acción.

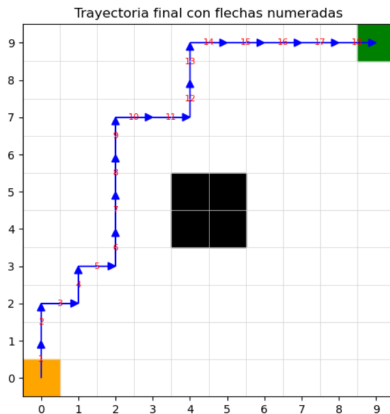
Ejemplo de Q-table

Estado	Arriba	Abajo	Izquierda	Derecha
s_1	0.3	0.1	0.2	0.8
s_2	0.5	0.4	0.1	0.2
s_3	0.2	0.6	0.3	0.4

- ▶ Por ejemplo, en el estado s_1 la acción con mayor valor Q es **Derecha**, por lo que el agente preferirá esa acción.
- ▶ A medida que el agente aprende mediante **exploración y explotación**, estos valores se actualizan y la tabla converge hacia las mejores decisiones.

Ejemplo:

- Un robot debe ir desde una posición inicial (x_0, y_0) hasta una meta (x_1, y_1) en un grid 2D, evitando obstáculos.



Ver código python.

SARSA (State-Action-Reward-State-Action)

SARSA (State-Action-Reward-State-Action)^a es un algoritmo de **aprendizaje por refuerzo basado en política (on-policy)** que permite a un agente aprender una política óptima mediante la interacción con su entorno.

- ▶ El agente explora el entorno.
- ▶ Ejecuta acciones.
- ▶ Recibe retroalimentación en forma de recompensas.
- ▶ Actualiza continuamente su comportamiento para **maximizar las recompensas a largo plazo**.

^aDiapositivas basadas en: [SARSA \(State-Action-Reward-State-Action\)](#) in Reinforcement Learning.

Características de SARSA

- ▶ A diferencia de algoritmos **off-policy** como **Q-Learning**, que aprenden a partir de las **mejores acciones posibles**, SARSA actualiza su conocimiento utilizando las **acciones reales ejecutadas por el agente**.
 - ▶ El aprendizaje depende directamente de la **secuencia real de interacción**:
$$(S, A, R, S', A')$$
 - ▶ Esto permite que el agente aprenda considerando su **comportamiento actual**.
- ▶ Por esta razón, SARSA es especialmente adecuado para entornos donde las acciones del agente y la retroalimentación inmediata **influyen directamente en el proceso de aprendizaje**.

Componentes del Algoritmo SARSA

- ▶ **Estado (S):** situación o posición actual del agente dentro del entorno.
- ▶ **Acción (A):** decisión o movimiento que el agente realiza en un estado determinado.
- ▶ **Recompensa (R):** retroalimentación inmediata que recibe el agente después de ejecutar una acción.
- ▶ **Siguiente estado (S'):** estado al que el agente transita después de realizar una acción.
- ▶ **Siguiente acción (A'):** acción que el agente ejecutará en el nuevo estado según su política actual.

Actualización de los Valores Q en SARSA

- ▶ El algoritmo **SARSA** actualiza los **valores Q** (que miden la calidad de un par estado–acción) considerando:
 - ▶ la **recompensa inmediata**, y
 - ▶ las **recompensas futuras esperadas**.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

▶ Donde:

- ▶ $Q(s_t, a_t)$: valor Q actual para el par estado–acción en el instante t .
- ▶ α : **tasa de aprendizaje** (entre 0 y 1) que determina cuánto se actualizan los valores Q.
- ▶ r_{t+1} : recompensa inmediata recibida después de ejecutar la acción a_t en el estado s_t .
- ▶ γ : **factor de descuento** (entre 0 y 1) que representa la importancia de las recompensas futuras.
- ▶ $Q(s_{t+1}, a_{t+1})$: valor Q asociado al siguiente par estado–acción.

Interpretación de la Actualización

Recompensa inmediata

- ▶ El agente recibe la recompensa r_{t+1} después de ejecutar la acción a_t en el estado s_t .

Recompensa futura esperada

- ▶ El término $Q(s_{t+1}, a_{t+1})$ permite estimar las recompensas futuras desde el siguiente estado.

Corrección del valor Q

- ▶ El valor Q se ajusta según la diferencia entre:
 - ▶ la recompensa esperada, y
 - ▶ la recompensa realmente obtenida.

Este proceso permite que el agente **mejore progresivamente sus decisiones** durante el aprendizaje.

Pasos del Algoritmo SARSA

1. Inicialización de los valores Q

- ▶ Se inicializan los valores de la **tabla Q** con valores arbitrarios para cada par estado–acción.

2. Selección del estado inicial

- ▶ El agente comienza en un estado inicial s_0 dentro del entorno.

3. Bucle de episodios

- ▶ Para cada episodio se establece el estado inicial s_t .
- ▶ Luego se selecciona una acción a_t según una política, generalmente ϵ -greedy.

4. Bucle de pasos dentro del episodio

Para cada paso del episodio se realizan las siguientes operaciones:

- ▶ Ejecutar la acción a_t .
- ▶ Observar la recompensa R_{t+1} y el nuevo estado s_{t+1} .
- ▶ Seleccionar la siguiente acción a_{t+1} según la política en el estado s_{t+1} .
- ▶ Actualizar el valor Q del par (s_t, a_t) utilizando la regla de actualización de SARSA.
- ▶ Actualizar las variables del proceso:

$$s_t = s_{t+1}, \quad a_t = a_{t+1}$$

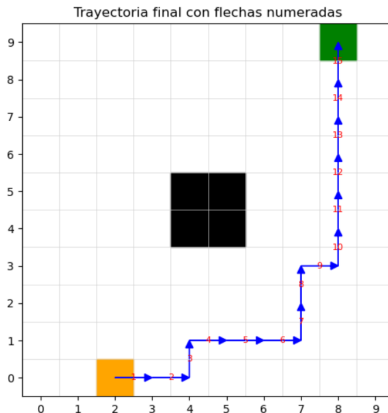
5. Finalización del episodio

- ▶ El proceso continúa repitiendo los pasos anteriores.
- ▶ El episodio termina cuando ocurre alguna de las siguientes condiciones:
 - ▶ El agente alcanza un **estado terminal**.
 - ▶ Se alcanza un **número máximo de pasos**.

Después de múltiples episodios de interacción con el entorno, el agente mejora progresivamente su política y aprende a tomar **decisiones cada vez más eficientes**.

Ejemplo:

- Un robot debe ir desde una posición inicial (x_0, y_0) hasta una meta (x_1, y_1) en un grid 2D, evitando obstáculos.



Ver código python.

Deep Q-Network.

El **Deep Q-Learning**^a es un método de **aprendizaje por refuerzo** que utiliza **redes neuronales profundas** para ayudar a los agentes a tomar decisiones en entornos complejos.

- ▶ Combina el algoritmo **Q-Learning** con técnicas de **aprendizaje profundo**.
- ▶ Permite aproximar la **función de valor de acción** cuando el espacio de estados es muy grande.
- ▶ Es especialmente útil cuando el número de posibles **estados** es muy elevado.

^aDiapositivas basadas en: [Deep Q-Learning in Reinforcement Learning](#).

Desafíos Abordados por Deep Q-Learning

- ▶ **Espacios de estados de alta dimensión**
 - ▶ El Q-Learning clásico utiliza una **tabla Q** para almacenar los valores.
 - ▶ Cuando el número de estados es muy grande, esta representación se vuelve **impracticable**.
 - ▶ Las **redes neuronales** permiten aproximar la función $Q(s, a)$ y manejar **gran cantidad de estados**.
- ▶ **Datos de entrada continuos**
 - ▶ Muchos problemas reales incluyen **datos continuos**, como **imágenes o señales**.
 - ▶ Las redes neuronales son especialmente adecuadas para **procesar este tipo de información**.
- ▶ **Escalabilidad**
 - ▶ El aprendizaje profundo permite que Q-Learning **escale a problemas más grandes**.
 - ▶ Esto hace posible resolver tareas que el **Q-Learning tabular tradicional** no puede manejar.

Arquitectura de Deep Q-Networks (DQN)

Una **Deep Q-Network (DQN)** está compuesta por varios elementos que permiten aproximar la función de valor de acción mediante aprendizaje profundo.

► Red Neuronal – Red principal (Online Network)

- La red principal es la que se entrena continuamente y se usa para aproximar la función de valor:

$$Q(s, a; \theta)$$

- θ representa los **parámetros entrenables** de la red.
- La red recibe como entrada el **estado del entorno**.
- Produce como salida un **vector de valores Q** para cada acción posible.
- Por ejemplo, en videojuegos como Atari la entrada puede ser **la imagen de la pantalla (píxeles)**.

► Arquitectura de DQN

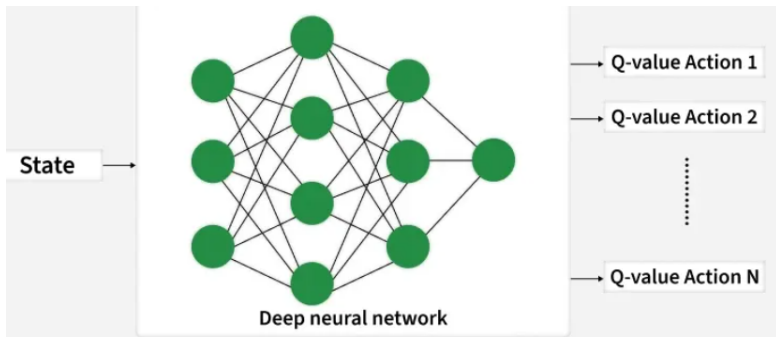


Figura tomada de: [Internet](#).

Experience Replay

- ▶ Para estabilizar el entrenamiento, las experiencias se almacenan en un **buffer de memoria**.
- ▶ Cada experiencia tiene la forma:

$$(s, a, r, s')$$

- ▶ Durante el entrenamiento se seleccionan **mini-lotes aleatorios**.
- ▶ Esto permite:
 - ▶ **romper la correlación** entre experiencias consecutivas
 - ▶ mejorar la **generalización del aprendizaje**

Red objetivo (Target Network)

- ▶ Se utiliza una **red objetivo** independiente, con la misma arquitectura que la red principal, con parámetros:

$$\theta^{-}$$

- ▶ Esta red se emplea para calcular los **valores Q objetivo**.

$$Q = r + \gamma \max_{a'} Q(s', a'; \theta^{-})$$

donde:

- ▶ θ^{-} son los parámetros de la red objetivo.
- ▶ r es la recompensa.
- ▶ γ es el factor de descuento.
- ▶ s' es el siguiente estado.
- ▶ Sus parámetros permanecen fijos durante varios pasos y **se actualizan periódicamente** con los de la red principal.
- ▶ Esto contribuye a **mejorar la estabilidad del entrenamiento**.

Función de Pérdida en DQN

El entrenamiento de la red se realiza minimizando una **función de pérdida** que mide la diferencia entre los valores Q predichos y los valores objetivo.

$$L(\theta) = E \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

- ▶ r : **recompensa inmediata**.
- ▶ γ : **factor de descuento**.
- ▶ $Q(s, a; \theta)$: valor Q estimado por la **red principal**.
- ▶ $Q(s', a'; \theta^-)$: valor Q estimado por la **red objetivo**.
- ▶ $\max_{a'}$: selección de la **mejor acción posible** en el estado siguiente.
- ▶ El objetivo del entrenamiento es **reducir el error cuadrático** entre el valor predicho y el valor objetivo.

Proceso de Entrenamiento de una Deep Q-Network

El entrenamiento de una **Deep Q-Network (DQN)** sigue una secuencia de pasos que permiten al agente aprender de la interacción con el entorno.

► Inicialización

- Se inicializa el **replay buffer**.
 - Inicialmente, el buffer de experiencias está vacío: $\mathcal{D} = \emptyset$ y se establece una capacidad máxima: $|\mathcal{D}| \leq N$
 - Cuando el buffer alcanza su capacidad máxima, las experiencias más antiguas se reemplazan por las nuevas.
- Se inicializan la **red principal** con parámetros θ y la **red objetivo** con parámetros θ^- .
- Se definen los **hiperparámetros** del modelo:
 - tasa de aprendizaje α
 - factor de descuento γ
 - tasa de exploración ϵ

Exploración vs. Explotación

- ▶ Se utiliza una política **ϵ -greedy** para equilibrar exploración y explotación.
- ▶ Con probabilidad ϵ , el agente selecciona una **acción aleatoria** (exploración).
- ▶ Con probabilidad $1 - \epsilon$, elige la acción con el **mayor valor Q** estimado.

Recolección de Experiencias

- ▶ El agente interactúa con el entorno y genera transiciones:
 (s, a, r, s')
- ▶ Estas experiencias se almacenan en el **replay buffer**.

Actualización del Modelo

- ▶ Se selecciona un **mini-batch aleatorio** del replay buffer.
- ▶ Se calculan los **valores Q objetivo** utilizando la red objetivo.
- ▶ La red principal se actualiza **minimizando la función de pérdida** mediante **descenso por gradiente**.

Actualización de la Red Objetivo

- ▶ Periódicamente se copian los **pesos de la red principal** a la red objetivo:

$$\theta^- \leftarrow \theta$$

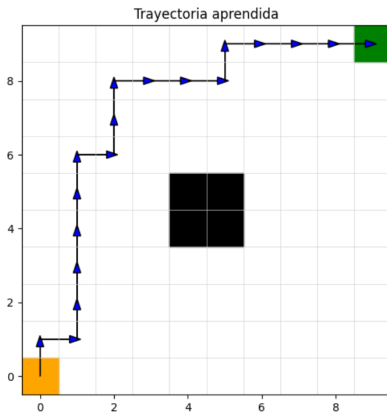
- ▶ Este procedimiento ayuda a **estabilizar el proceso de aprendizaje**.

Disminución de la Exploración

- ▶ La tasa de exploración ϵ se **reduce gradualmente durante el entrenamiento**.
- ▶ Esto permite que el agente pase de:
 - ▶ **explorar el entorno** al inicio
 - ▶ a **explotar la política aprendida** posteriormente.

Ejemplo:

- ▶ Un robot debe ir desde una posición inicial (x_0, y_0) hasta una meta (x_1, y_1) en un grid 2D, evitando obstáculos.



Ver código python.

Arquitectura Dyna-Q.

- ▶ La arquitectura *Dyna*^a combina métodos **model-free** y **model-based** para aprender políticas óptimas en problemas de aprendizaje por refuerzo.

^aDiapositivas basadas en: [Reinforcement Learning: Dyna Algorithm](#).

Planificación en la Arquitectura Dyna

- ▶ Supóngase que las acciones solo pueden ejecutarse en **instantes específicos de tiempo**, por lo que existe un cierto **tiempo de espera** entre acciones.
- ▶ Durante ese intervalo se pueden realizar **acciones simuladas** (no acciones reales en el entorno) y actualizar la política utilizando el valor de retorno obtenido.
- ▶ Este proceso se conoce como **planificación** (*planning*).

Dinámica del Entorno y Ecuaciones de Bellman

Las ecuaciones de Bellman utilizan la dinámica del entorno, es decir, información sobre cómo evoluciona el entorno cuando el agente ejecuta una acción.

Componentes de la dinámica del entorno

- **Probabilidad de transición**

$$P(s'|s, a)$$

Probabilidad de llegar al estado s' cuando el agente se encuentra en s y ejecuta la acción a .

- **Recompensa esperada**

$$R(s, a, s')$$

Recompensa obtenida al pasar del estado s al estado s' después de ejecutar la acción a .

Ejemplo conceptual

- ▶ Un robot en el estado s ejecuta la acción *moverse a la derecha*:
 - ▶ Con probabilidad 0.8 llega al estado s_1
 - ▶ Con probabilidad 0.2 llega al estado s_2
- ▶ Cada transición produce una recompensa asociada.
- ▶ Las ecuaciones de Bellman utilizan la dinámica del entorno para calcular el **valor esperado de un estado** bajo una política π

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) [r + \gamma V^{\pi}(s')]$$

- ▶ $p(s', r|s, a)$ describe la **dinámica del entorno**.
- ▶ Representa la probabilidad de llegar al estado s' y recibir una recompensa r al ejecutar la acción a en el estado s .
- ▶ Esta información permite calcular el **retorno esperado futuro**.

Aprendizaje, Planificación y Acción

- ▶ La arquitectura *Dyna* proporciona un marco general que integra tres procesos fundamentales: **aprendizaje**, **planificación** y **acción**.



Figura tomada de: [Internet](#).

- ▶ **Aprendizaje del modelo (Model Learning)** El agente utiliza sus interacciones con el entorno para aprender un **modelo** que aproxima la dinámica del entorno.
- ▶ **Planificación (Planning)** El agente realiza planificación utilizando el modelo aprendido para **simular estados futuros y recompensas**. Esto permite evaluar diferentes acciones y mejorar la política **sin ejecutarlas realmente en el entorno**.
- ▶ **Acción (Acting)** El agente ejecuta acciones en el **entorno real** de acuerdo con su política actual.

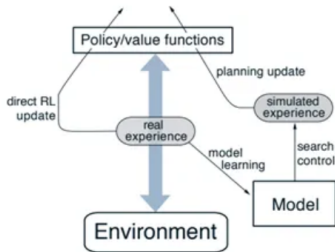


Figura tomada de: [Internet](#).

Funcionamiento General

- ▶ En la arquitectura *Dyna*, el agente interactúa con el entorno, actualiza su modelo, realiza planificación utilizando dicho modelo y ajusta su política a partir de los valores aprendidos.

Pseudocódigo del algoritmo Dyna.

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \epsilon$ -greedy(S, Q)
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

Model Learning

Planning

Figura tomada de: [Internet](#).

Paso 1: Inicialización

- ▶ Se inicializan aleatoriamente:
 - ▶ Los **valores estado-acción**.
 - ▶ El **modelo del entorno** (dinámica del ambiente).

Paso 2: Aprendizaje del modelo (RL directo)

- ▶ El agente interactúa con el entorno seleccionando acciones y observando las **transiciones de estado** y las **recompensas** obtenidas.
- ▶ Por ejemplo, si el agente ejecuta la acción *right* desde el estado (1, 1), puede observar una transición al estado (1, 2) con recompensa 0.
- ▶ A partir de estas observaciones, el agente **actualiza su modelo del entorno**, registrando las transiciones de estado y las recompensas observadas.

Paso 3: Planificación

La fase de planificación dentro de la arquitectura *Dyna* está compuesta por tres elementos principales:

- ▶ **Control de búsqueda (Search Control)**
 - ▶ Se selecciona aleatoriamente un par **estado–acción** previamente visitado.
 - ▶ Por ejemplo, durante la planificación el agente puede simular ejecutar la acción *up* desde el estado (2, 2) y observar que alcanza el estado objetivo con una recompensa de 1.
- ▶ **Consulta del modelo (Model Query)**
 - ▶ Utilizando el modelo aprendido, el agente realiza planificación para **simular o predecir estados futuros y recompensas**.
- ▶ **Actualización de valores (Value Updates)**
 - ▶ A partir de la experiencia simulada, el agente dispone de la transición (S, A, R, S').
 - ▶ Con esta información puede aplicar algoritmos como *Monte Carlo (MC)*, *Temporal Difference (TD)* o *Programación Dinámica (DP)* para actualizar su **función de valor**.

Paso 4: Acción

- ▶ El agente selecciona acciones para ejecutarlas en el **entorno real** de acuerdo con su **política actual**.
- ▶ Inicialmente, la política puede ser **aleatoria o exploratoria**.
- ▶ A medida que el agente obtiene más experiencias reales y realiza procesos de planificación, la política se **actualiza progresivamente** para mejorar la toma de decisiones.

Observación

La **política se actualiza** tanto durante la fase de **aprendizaje directo (Direct RL)** como durante la fase de **planificación**.

Limitación

Este enfoque puede fallar si el **modelo no es estático**; es decir, si las recompensas asociadas a los pares **estado–acción** cambian con el tiempo.

Dyna-Q+: Exploración en Entornos Cambiantes

El algoritmo *Dyna-Q+* introduce un mecanismo para manejar entornos donde el modelo puede cambiar con el tiempo.

- ▶ En este enfoque, la **recompensa se ajusta en función del tiempo** transcurrido desde la última visita a un par estado–acción.
- ▶ Si un estado–acción no ha sido visitado durante mucho tiempo, el algoritmo **incrementa su recompensa**.

- ▶ Este incremento es proporcional a la **raíz cuadrada del número de pasos de tiempo**, lo que incentiva al agente a **explorar nuevamente estados antiguos**.

$$R' = R + K\sqrt{\tau}$$

- ▶ τ representa el número de **pasos de tiempo desde la última vez que se visitó la transición**.
- ▶ K es una **constante** que controla cuánto peso se da a este término de exploración.
- ▶ Si $K = 0$, el algoritmo **Dyna-Q+** se reduce al algoritmo **Dyna-Q** estándar.

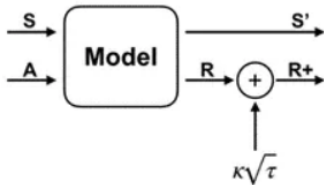
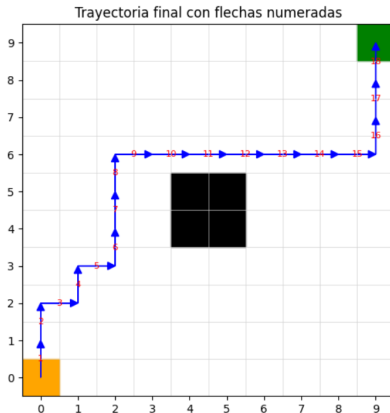


Figura tomada de: [Internet](#).

Ejemplo:

- Un robot debe ir desde una posición inicial (x_0, y_0) hasta una meta (x_1, y_1) en un grid 2D, evitando obstáculos.



Ver código python.

REINFORCE.

El algoritmo **REINFORCE^a** es un método de **aprendizaje por refuerzo basado en gradiente de política** que busca mejorar la toma de decisiones de un agente.

- ▶ El agente aprende **probando acciones** y ajusta la **probabilidad de seleccionarl**as según el **retorno obtenido**.
- ▶ A diferencia de los métodos basados en valores, **no estima funciones $V(s)$ ni $Q(s, a)$** .
- ▶ En su lugar, **aprende directamente la política óptima** optimizando sus parámetros mediante **gradiente de política**.
- ▶ Constituye una de las **bases fundamentales de los métodos modernos de Policy Gradient**.

^aDiapositivas basadas en: [Policy Gradient Methods and REINFORCE Algorithm](#). y [REINFORCE Algorithm](#).

Comparación con Deep Q-Learning

- ▶ **DQN** aprende una función de valor $Q(s, a)$ y deriva una política codiciosa:

$$\pi(s) = \arg \max_a Q(s, a)$$

- ▶ **REINFORCE**, en cambio, **aprende directamente la política** sin depender de funciones de valor intermedias.

Características principales

- ▶ Pertenece a la familia de métodos de **Policy Gradient**.
- ▶ Resulta adecuado cuando existen **muchas acciones posibles**.
- ▶ También es útil cuando las acciones pertenecen a **espacios continuos**.
- ▶ Es especialmente conveniente cuando es **difícil estimar el valor de cada acción** mediante funciones de valor.

Funcionamiento del Algoritmo REINFORCE

El algoritmo **REINFORCE** aprende una política mediante actualizaciones basadas en el retorno obtenido en cada episodio.

1. Recolección de Episodios

- ▶ El agente interactúa con el entorno siguiendo la **política actual**.
- ▶ Esta interacción genera una **trayectoria** formada por:

$$(s_0, a_0, r_0, s_1, a_1, r_1, \dots)$$

- ▶ La trayectoria incluye **estados, acciones y recompensas**.

2. Cálculo del Retorno

- ▶ Para cada instante t se calcula el **retorno acumulado** G_t .
- ▶ Generalmente se utiliza la **suma descontada de recompensas**:

$$G_t = \sum_{k=t}^T \gamma^{k-t} R_k$$

- ▶ donde:
 - ▶ γ : **factor de descuento**
 - ▶ T : último paso temporal del episodio
 - ▶ R_k : recompensa recibida en el instante k

3. Actualización mediante Gradiente de Política

- ▶ Los parámetros de la política θ se actualizan utilizando:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$$

- ▶ donde:

- ▶ α : **tasa de aprendizaje**.
- ▶ $\pi_{\theta}(a_t | s_t)$: probabilidad de seleccionar la acción a_t en el estado s_t .
- ▶ G_t : **retorno acumulado** desde el instante t .
- ▶ $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$: gradiente que indica cómo ajustar la probabilidad de la acción.

4. Repetición del Proceso

- ▶ El procedimiento se repite durante **múltiples episodios**.
- ▶ En cada iteración la política se ajusta en dirección de **mayores recompensas esperadas**.
- ▶ De esta manera, el agente aprende progresivamente una **política más efectiva**.

Variantes del Algoritmo REINFORCE

Debido a la **alta varianza** del algoritmo REINFORCE, se han propuesto diversas modificaciones para mejorar su estabilidad.

REINFORCE con Baseline

- ▶ Una estrategia común consiste en restar un **valor baseline** al retorno.
- ▶ Generalmente se utiliza la **función de valor del estado** $V(s)$.
- ▶ Esta técnica **reduce la varianza** del estimador del gradiente sin modificar su valor esperado.

La regla de actualización se expresa como:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b_t)$$

- ▶ b_t : valor baseline, por ejemplo $V(s_t)$.
- ▶ G_t : retorno acumulado desde el instante t .

Otra extensión importante del algoritmo REINFORCE es el enfoque **Actor–Critic**.

- ▶ El método se basa en dos componentes:
 - ▶ **Actor**: selecciona las acciones según la política.
 - ▶ **Critic**: evalúa la calidad de dichas acciones.
- ▶ El Critic proporciona **retroalimentación** al Actor mediante una estimación del valor.
- ▶ Esto permite:
 - ▶ **reducir la varianza**
 - ▶ mejorar la **estabilidad del aprendizaje**
 - ▶ acelerar la **convergencia**.

Ventajas del Algoritmo REINFORCE

- ▶ **Simplicidad conceptual**
 - ▶ Es un algoritmo **fácil de entender e implementar**.
 - ▶ Constituye un buen punto de partida para estudiar **Policy Gradient**.
- ▶ **Optimización directa de la política**
 - ▶ El algoritmo mejora **directamente la forma de seleccionar acciones**.
 - ▶ Resulta adecuado cuando existen **muchas acciones posibles**.
- ▶ **Adecuado para tareas episódicas**
 - ▶ Funciona bien en problemas donde existe un **final claro del episodio**.
 - ▶ En estos casos el agente recibe una **recompensa acumulada al final**.

Desafíos del Algoritmo REINFORCE

A pesar de su simplicidad, el algoritmo presenta varias limitaciones.

▶ Alta varianza

- ▶ El gradiente se estima a partir de **una sola trayectoria**.
- ▶ El retorno G_t puede variar significativamente entre episodios.
- ▶ Esto genera un proceso de aprendizaje **ruidoso y lento**.

▶ Ineficiencia en el uso de muestras

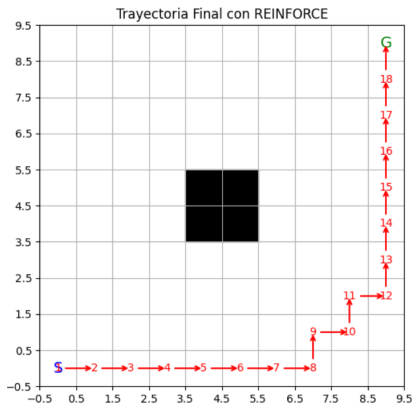
- ▶ REINFORCE requiere **episodios completos** para actualizar la política.
- ▶ El agente puede necesitar **muchas interacciones** con el entorno para aprender.

▶ Problemas de convergencia

- ▶ La alta variabilidad puede dificultar la **convergencia**.
- ▶ Se requieren **muchos episodios de entrenamiento** para aprender una política adecuada.

Ejemplo:

- Un robot debe ir desde una posición inicial (x_0, y_0) hasta una meta (x_1, y_1) en un grid 2D, evitando obstáculos.



Ver código python.

Actor-Critic.

El algoritmo **Actor-Critic**^a es un método de **aprendizaje por refuerzo** que combina dos componentes principales:

- ▶ **Actor**: encargado de **seleccionar las acciones**.
- ▶ **Critic**: encargado de **evaluar la calidad de dichas acciones**.
- ▶ El **Actor aprende a tomar decisiones**, mientras que el **Critic evalúa qué tan buenas son**.
- ▶ La interacción entre ambos permite **equilibrar exploración y aprendizaje**.
- ▶ Como resultado, el proceso de aprendizaje se vuelve **más estable y balanceado**.

^aDiapositivas basadas en: [Actor-Critic Algorithm in Reinforcement Learning](#).

Conceptos Clave

El método Actor–Critic se basa en dos conceptos fundamentales:

► Política (Actor)

- La política se denota como: $\pi(a|s)$
- Representa la **probabilidad de seleccionar la acción a en el estado s** .
- El objetivo del actor es **maximizar el retorno esperado**.
- La política es modelada mediante una **red neuronal del actor**.
- Sus parámetros se denotan por: θ

► Función de Valor (Critic)

- La función de valor se denota como: $V(s)$
- Estima la **recompensa acumulada esperada** desde el estado s .
- Es modelada mediante la **red neuronal del crítico**.
- Sus parámetros se representan por: w

Funcionamiento del Algoritmo Actor–Critic

El algoritmo **Actor–Critic** combina dos procesos de aprendizaje:

- ▶ **Actor**: aprende la **política** que determina qué acción ejecutar.
- ▶ **Critic**: aprende una **función de valor** que evalúa la calidad de las acciones tomadas.

El objetivo de este método es **optimizar simultáneamente**:

- ▶ La **política** del agente (Actor).
- ▶ La **estimación del valor** de los estados o acciones (Critic).
- ▶ El **Actor** actualiza la política usando **gradiente de política**.
- ▶ El **Critic** aprende una **estimación del valor** que sirve como retroalimentación para el Actor.
- ▶ La función objetivo total combina **dos componentes principales**.

Función Objetivo del Actor

La actualización del **Actor** se basa en el **gradiente de la política**:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \nabla_{\theta} \log \pi_{\theta}(a_i | s_i) A(s_i, a_i)$$

Interpretación de los términos:

- ▶ $J(\theta)$: **retorno esperado** bajo la política parametrizada por θ .
- ▶ $\pi_{\theta}(a|s)$: **política** que define la probabilidad de ejecutar la acción a en el estado s .
- ▶ N : número de **experiencias o muestras**.
- ▶ $A(s, a)$: **función de ventaja**, que indica qué tan beneficiosa es una acción en un estado.
- ▶ i : índice de la muestra utilizada en el entrenamiento.
- ▶ El Actor busca **maximizar la recompensa esperada** ajustando los parámetros θ .

Actualización de la Función de Valor (Critic)

El **Critic** aprende una estimación del valor minimizando el error entre valores estimados.

$$\nabla_w J(w) \approx \frac{1}{N} \sum_{i=1}^N \nabla_w (V_w(s_i) - Q_w(s_i, a_i))^2$$

Significado de los términos:

- ▶ $\nabla_w J(w)$: gradiente de la **función de pérdida** respecto a los parámetros w .
- ▶ N : número de **muestras utilizadas**.
- ▶ $V_w(s_i)$: estimación del **valor del estado** realizada por el Critic.
- ▶ $Q_w(s_i, a_i)$: estimación del **valor de acción**.
- ▶ i : índice de la muestra utilizada en el aprendizaje.
- ▶ El Critic busca **reducir el error de estimación del valor**.

Reglas de Actualización del Actor y el Critic

Los parámetros del Actor y del Critic se actualizan mediante métodos de optimización basados en gradientes.

Actualización del Actor

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta_t)$$

- ▶ α : **tasa de aprendizaje del Actor.**
- ▶ t : paso temporal dentro de un episodio.
- ▶ El Actor utiliza **gradient ascent** para **maximizar la recompensa esperada.**

Actualización del Critic

$$w_t = w_t - \beta \nabla_w J(w_t)$$

- ▶ w : parámetros de la **red del Critic.**
- ▶ β : **tasa de aprendizaje del Critic.**
- ▶ El Critic utiliza **gradient descent** para **minimizar el error de estimación.**

Función de Ventaja (Advantage Function)

La **función de ventaja** mide qué tan beneficiosa es una acción respecto al valor promedio esperado de un estado bajo la política actual.

- ▶ Se denota como:

$$A(s, a)$$

- ▶ Permite determinar **cuánto mejor o peor** es ejecutar la acción a en el estado s en comparación con el comportamiento promedio de la política.

Definición matemática

$$A(s, a) = Q(s, a) - V(s)$$

- ▶ $Q(s, a)$: **valor de acción**, recompensa esperada al ejecutar la acción a en el estado s .
- ▶ $V(s)$: **valor del estado**, recompensa esperada promedio al seguir la política desde s .

Interpretación de la Función de Ventaja

La función de ventaja proporciona una medida relativa de la calidad de una acción:

- ▶ Si $A(s, a) > 0$, la acción es **mejor que el promedio**.
- ▶ Si $A(s, a) < 0$, la acción es **peor que el promedio**.
- ▶ Si $A(s, a) = 0$, la acción tiene un **rendimiento promedio**.

En el método **Actor–Critic**:

- ▶ El **Actor** se actualiza utilizando el **gradiente de la política**, favoreciendo acciones con **mayor ventaja**.
- ▶ El **Critic** se actualiza para **minimizar la diferencia** entre:
 - ▶ el **valor estimado del estado**
 - ▶ y el **valor de la acción**.
- ▶ Estas operaciones constituyen los **cálculos fundamentales del método Actor–Critic**.

► Función de Ventaja

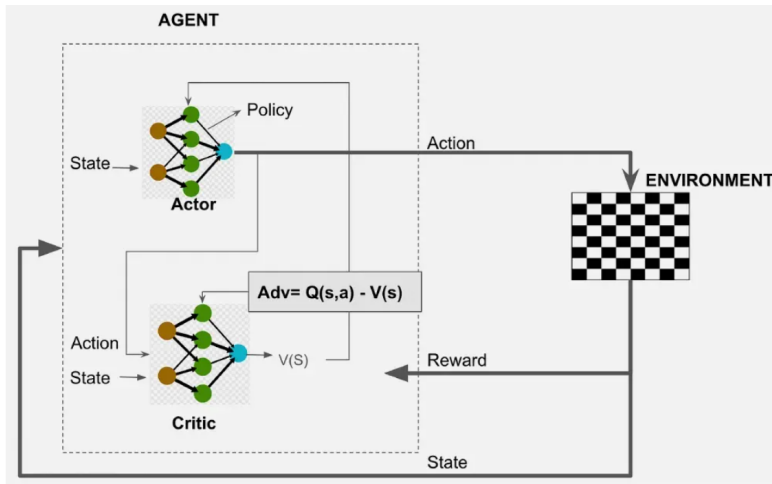


Figura tomada de: [Internet](#).

Ventajas del Método Actor–Critic

- ▶ **Mayor eficiencia de muestreo**
 - ▶ La naturaleza híbrida del algoritmo permite aprender con **menos interacciones con el entorno**.
 - ▶ Aprendizaje **más eficiente en el uso de los datos**.
- ▶ **Convergencia más rápida**
 - ▶ La política y la función de valor se actualizan **de forma simultánea**.
 - ▶ Esto permite una **adaptación más rápida** durante el proceso de entrenamiento.
- ▶ **Flexibilidad en espacios de acción**
 - ▶ La arquitectura Actor–Critic puede trabajar con:
 - ▶ **espacios de acción discretos**
 - ▶ **espacios de acción continuos**
 - ▶ Permitiendo abordar una **gran variedad de problemas de RL**.
- ▶ **Aprendizaje Off-Policy (en algunas variantes)**
 - ▶ Algunas variantes pueden **aprender a partir de experiencias pasadas**.
 - ▶ No es necesario seguir estrictamente la **política actual**.

Variantes del Algoritmo Actor–Critic

Advantage Actor–Critic (A2C)

- ▶ Modifica el Critic para estimar la **función de ventaja**.
- ▶ La función de ventaja mide qué tan buena es una acción respecto al promedio.

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

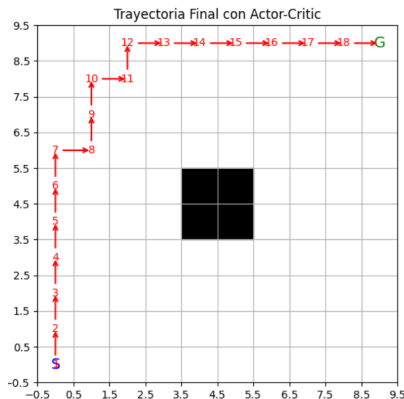
- ▶ Esta formulación ayuda a **reducir la varianza del gradiente de la política**.
- ▶ Como resultado, se obtiene un **aprendizaje más estable y eficiente**.

Asynchronous Advantage Actor–Critic (A3C)

- ▶ Extiende A2C utilizando **múltiples agentes ejecutándose en paralelo**.
- ▶ Cada agente interactúa con su propio entorno y **actualiza la política de forma asíncrona**.
- ▶ Esto reduce la **correlación entre actualizaciones** y mejora la **estabilidad del aprendizaje**.
- ▶ Además, permite un **entrenamiento más rápido**.

Ejemplo:

- Un robot debe ir desde una posición inicial (x_0, y_0) hasta una meta (x_1, y_1) en un grid 2D, evitando obstáculos.



Ver código python.

Evaluación de las técnicas de aprendizaje por refuerzo (RL)

La evaluación de las técnicas de aprendizaje por refuerzo (RL) se realiza considerando varios criterios que miden la eficacia, eficiencia y estabilidad del agente a lo largo del aprendizaje.

Recompensa acumulada.

- ▶ La métrica más directa: cuánta **recompensa total** obtiene el agente en un episodio o conjunto de episodios.
- ▶ Recompensa acumulada en un episodio que dura T pasos:

$$R = \sum_{t=0}^T r_t$$

- ▶ En problemas con descuento ($\gamma \in [0, 1]$):

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$$

- ▶ Permite comparar distintas políticas o algoritmos y ver cuál obtiene más retorno.

Convergencia de la política y función de valor.

- ▶ Se evalúa si la función de valor $Q(s, a)$ o la política $\pi(a|s)$ **alcanzan valores estables tras varias iteraciones.**
- ▶ Una política estable y consistente indica aprendizaje exitoso.
- ▶ Se pueden graficar los valores de Q a lo largo del tiempo para observar estabilidad y convergencia.

Velocidad de aprendizaje y eficiencia de muestras.

- ▶ Mide **cuántos pasos o episodios necesita un algoritmo** para alcanzar una política cercana a óptima.
- ▶ Algoritmos más eficientes requieren menos interacciones con el entorno.
- ▶ Permite comparar distintos métodos y seleccionar el más rápido en entornos complejos.

Robustez y estabilidad.

- ▶ Evaluar si la política aprendida sigue funcionando bien ante pequeñas variaciones del entorno.
- ▶ Se realizan experimentos con diferentes semillas aleatorias para verificar consistencia.

Exploración versus explotación.

- ▶ Verificar que el agente explora acciones nuevas y no queda atrapado en óptimos locales.
- ▶ Métodos con buena exploración tienden a encontrar soluciones cercanas a óptimas.

Visualización y seguimiento.

- ▶ Gráficas de métricas a lo largo del tiempo:
 - ▶ Recompensa por episodio.
 - ▶ Error temporal-diferencia (TD error).
 - ▶ Valores estimados de estados o acciones.

Permite identificar problemas como divergencia, aprendizaje lento o sobreajuste.

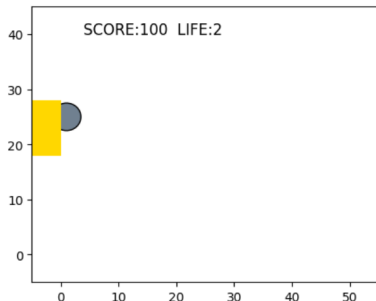
Resumen de Aprendizaje por Refuerzo

- ▶ El aprendizaje por refuerzo combina exploración y explotación.
- ▶ Su objetivo es maximizar la recompensa acumulada a largo plazo.
- ▶ Se modela matemáticamente como un Proceso de Decisión de Markov.
- ▶ Las funciones de valor (V , Q) permiten evaluar y mejorar políticas.

Taller: Juegos con Aprendizaje por Refuerzo

- ▶ Estudiar y mejorar, empleando técnicas de RL, los siguientes juegos.

Juego de Pong



Juego Triqui



Ver código python.

Gracias por su atención.

